Smart Travel Planner

Final System Appearance and Data Linking Flow

At the end, the system should look like a Smart Travel Planner dashboard where one saved trip becomes the main record, and all external APIs attach extra information to that same trip.

```
User creates 1 trip
      ↓
Trip saved in your own database
      ↓
System uses trip destination / country / coordinates
      ↓
4 external APIs fetch extra travel information
      ↓
Frontend displays one combined travel summary
```

The app should combine your self-developed API with third-party API data, accept user input, combine the data, and display the result clearly. This plan uses four third-party APIs so each member has one clear development ownership.

1. Final System Appearance

The final app can have these main pages:

```
1. Home / Trip List Page
2. Add Trip Page
3. Trip Details / Smart Travel Summary Page
4. Edit Trip Page
5. Testing / API Demo through Postman
```

The most important page is the Trip Details / Smart Travel Summary Page.

```
Smart Travel Planner

Trip: Tokyo, Japan
Date: 12 July 2026 - 18 July 2026
Preference: Culture, Food
Notes: Visit cultural places and try local restaurants.

Weather
- Temperature: 27°C
- Condition: Cloudy
- Humidity: 70%
- Wind: 3.2 m/s

Google Places Nearby
1. Tokyo Tower
2. Shibuya Crossing
3. Ueno Park

Foursquare Recommendations
1. Senso-ji Temple
2. Tsukiji Market
3. Local ramen restaurant

Country Information
- Country: Japan
- Capital: Tokyo
- Currency: Japanese Yen
- Language: Japanese
- Region: Asia
- Flag: Japan
```

The system should not show four separate unrelated API results. It should show one combined travel result based on one trip record.

2. Main Data Relationship

Your own database is the center of the system.

```
Trip Record
  ├── Weather Data from OpenWeatherMap
  ├── Nearby Places from Google Maps / Places
  ├── Recommendations from Foursquare
  └── Country Info from REST Countries
```

The Trip record is the parent. The external API results are temporary child data fetched using values from the trip.

3. Suggested Database Structure

You only need to permanently store the user-created trip data.

Trip Table / Collection

```
Trip {
  id: "trip_001",
  destinationName: "Tokyo",
  destinationCountry: "Japan",
  latitude: 35.6762,
  longitude: 139.6503,
  startDate: "2026-07-12",
  endDate: "2026-07-18",
  preferences: ["culture", "food"],
  travelNotes: "Visit cultural places and try local food",
  createdAt: "2026-05-20T10:00:00Z",
  updatedAt: "2026-05-20T10:00:00Z"
}
```

This one trip record contains the important linking fields:

Field	Used By
destinationName	Search/display trip name
destinationCountry	REST Countries API
latitude	OpenWeatherMap, Google Places, Foursquare
longitude	OpenWeatherMap, Google Places, Foursquare
preferences	Foursquare recommendation filtering
startDate, endDate	Display trip period
travelNotes	User's own travel record

The assignment requires user travel records, CRUD operations, database storage, JSON request/response, and endpoints like POST /api/trips, GET /api/trips, PUT /api/trips/{id}, and DELETE /api/trips/{id}.

4. How Each API Links to the Trip

A. OpenWeatherMap API

OpenWeatherMap uses latitude and longitude from the trip.

```
Trip.latitude + Trip.longitude → OpenWeatherMap → Weather Card
```

```
weather: {
  temperature: 27,
  condition: "Clouds",
  humidity: 70,
```

```
  windSpeed: 3.2
}
```

B. Google Maps / Places API

Google Places uses latitude, longitude, and place type. It is useful for nearby hotels, cafes, tourist attractions, and restaurants.

```
Trip.latitude + Trip.longitude → Google Places → Nearby Places Section
```

```
googlePlaces: [
  {
    name: "Tokyo Tower",
    address: "Tokyo, Japan",
    rating: 4.5,
    type: "tourist_attraction"
  }
]
```

C. Foursquare API

Foursquare uses latitude, longitude, and user preference. It should be used for preference-based recommendation lists.

User Preference	Foursquare Query
food	restaurant, cafe, local food
culture	museum, temple, landmark
nature	park, garden
shopping	mall, market
family	zoo, aquarium, attraction

```
Trip.latitude + Trip.longitude + Trip.preferences
      ↓
Foursquare
      ↓
Recommended Places Section
```

API	Purpose
Google Places	General nearby place search / map-related data
Foursquare	Preference-based recommendation list

D. REST Countries API

REST Countries uses the destinationCountry field from the trip.

```
Trip.destinationCountry → REST Countries → Country Info Card
```

```
countryInfo: {
  country: "Japan",
  capital: "Tokyo",
  currency: "Japanese Yen",
  languages: ["Japanese"],
  region: "Asia",
  flag: "Japan"
}
```

5. Final Data Flow

This is the cleanest final architecture:

```
Frontend
  |
  | user creates trip
```

```
  v
Backend API
  |
  | saves trip
  v
Database
  |
  | get trip by ID
  v
Backend Summary Service
  |
  | uses trip fields
  |--------------------------------------|
  |                                      |
  v                                      v
OpenWeatherMap                      Google Places
uses lat/lng                        uses lat/lng
  |                                      |
  v                                      v
Weather data                        Nearby places

  |--------------------------------------|
  |                                      |
  v                                      v
Foursquare                          REST Countries
uses lat/lng + preference           uses country name
  |                                      |
  v                                      v
Recommendations                     Country info

  |
  v
Combined JSON response
  |
  v
Frontend Summary Page
```

6. Final Combined Endpoint

To make the system feel connected, create one important endpoint:

```
GET /api/trips/:id/summary
```

This endpoint should:

```
1. Find trip from your database.
2. Read latitude, longitude, country, and preferences.
3. Call OpenWeatherMap.
4. Call Google Places.
5. Call Foursquare.
6. Call REST Countries.
7. Combine all results.
8. Return one JSON response.
```

Example final response:

```
{
  "trip": {
    "id": "trip_001",
    "destinationName": "Tokyo",
    "destinationCountry": "Japan",
    "startDate": "2026-07-12",
    "endDate": "2026-07-18",
    "preferences": ["culture", "food"],
    "travelNotes": "Visit cultural places and try local food"
  },
  "weather": {
    "temperature": 27,
    "condition": "Clouds",
    "humidity": 70,
    "windSpeed": 3.2
  },
  "googlePlaces": [
```

```
    {
      "name": "Tokyo Tower",
      "type": "tourist_attraction",
      "rating": 4.5,
      "address": "Tokyo, Japan"
    }
  ],
  "foursquareRecommendations": [
    {
      "name": "Senso-ji Temple",
      "category": "Temple",
      "distance": 3500,
      "address": "Asakusa, Tokyo"
    }
  ],
  "countryInfo": {
    "country": "Japan",
    "capital": "Tokyo",
    "currency": "Japanese Yen",
    "languages": ["Japanese"],
    "region": "Asia",
    "flag": "Japan"
  }
}
```

This endpoint is the heart of the project.

7. Frontend Page Layout

```
Smart Travel Planner

[Add New Trip]

Trip List:
- Tokyo, Japan
- Bangkok, Thailand
- Seoul, South Korea

When user clicks Tokyo:

Trip Summary Page

[Trip Details Card]
Destination: Tokyo, Japan
Date: 12 July - 18 July
Preference: Culture, Food
Notes: Visit cultural places and try local food

[Weather Card]
Temperature, condition, humidity, wind

[Nearby Places Card]
Google Places results

[Recommendation Card]
Foursquare results based on preference

[Country Info Card]
Capital, currency, language, region, flag

[Edit Trip] [Delete Trip] [Refresh External Data]
```

The key thing: the page should look like one travel planner, not like four separate mini-apps.

8. Should External API Data Be Stored?

Data Type	Store in Database?	Reason
Trip details	Yes	This is your self-developed API data

Data Type	Store in Database?	Reason
Weather	No	Weather changes often
Google Places	Optional, but not necessary	Can fetch live during summary
Foursquare recommendations	Optional, but not necessary	Can fetch live during summary
Country info	Optional cache	Country info rarely changes

The simplest design is: store only trip records, and fetch external data when viewing the summary.

9. How the 4 Persons' Work Joins Together

Person	Module	Input from Trip	Output to Summary
Person 1	Trip CRUD + Weather	lat, lng	trip data + weather
Person 2	Google Places	lat, lng	nearby places
Person 3	Foursquare	lat, lng, preferences	recommendations
Person 4	REST Countries + Summary	destinationCountry	country info + final combined response

Each person does not build a separate system. Each person builds one module that plugs into the combined summary endpoint.

```
GET /api/trips/:id/summary
```

10. Final System Should Have These Main Endpoints

```
Trip CRUD
POST   /api/trips
GET    /api/trips
GET    /api/trips/:id
PUT    /api/trips/:id
DELETE /api/trips/:id

External Data
GET    /api/trips/:id/weather
GET    /api/trips/:id/google-places
GET    /api/trips/:id/recommendations
GET    /api/trips/:id/country-info

Combined Output
GET    /api/trips/:id/summary

Optional Authentication
POST   /api/auth/register
POST   /api/auth/login
GET    /api/auth/profile
```

11. Simple Explanation for the Group

Our database stores the user's trip record. Each trip contains destination name, country, coordinates, dates, notes, and travel preferences. When the user opens the trip summary page, the backend reads the saved trip record and uses its fields to call four external APIs. The coordinates are used to fetch weather from OpenWeatherMap, nearby places from Google Places, and recommendations from Foursquare. The country name is used to fetch country information from REST Countries. The backend then combines all results into one JSON response and the frontend displays it as a complete smart travel summary.